

Cortex Studio

Browser-Based C Development Environment
for Embedded ARM Targets

Architecture & Code Walkthrough

Structure · File & function reference · Data flow · Production workflow · Interview Q&A

Candidate: Tanishq Anoop Khandar

Flask · React · arm-none-eabi-gcc · Podman · AWS EC2 · Cloudflare Tunnel

1 What this project is

Cortex Studio is a web application that lets an authenticated user create a **workspace**, upload an **ECU description file (.a2l)**, write C code against the signals that file declares, compile it for an **ARM Cortex-M target** using the `arm-none-eabi-gcc` toolchain, and publish the resulting binary to a shared **marketplace**. The sharp edge of the problem is that it compiles code written by an authenticated stranger on the server, so every compile runs inside a locked-down, throwaway container.

It is the same class of tool as the “SDX Studio”-style editor in the screenshots: a signals panel on the left, a Monaco code editor in the middle, and Upload A2L / Save / Compile actions along the top.

Technology stack

Layer	Choice	Why
Frontend	React + Vite	SPA with react-router; Monaco as the in-browser C editor (not hand-rolled).
Backend	Flask (app factory + Blueprints)	One process serves both the built SPA and the JSON API.
Database	SQLite + Flask-Migrate	Simplicity over durability on a free-tier single box; Alembic migrations.
Compiler	arm-none-eabi-gcc	Debian's gcc-arm-none-eabi, run only inside a Podman sandbox.
Sandbox	Podman (rootless)	One short-lived, capability-dropped container per build.
Auth	Flask server-side sessions	HttpOnly signed cookie; passwords hashed with Werkzeug (PBKDF2).
Hosting	AWS EC2 t3.micro	gunicorn under systemd; Cloudflare Tunnel for public HTTPS.
CI/CD	GitHub Actions	Test on every push; auto-deploy to EC2 on green master.

High-level architecture

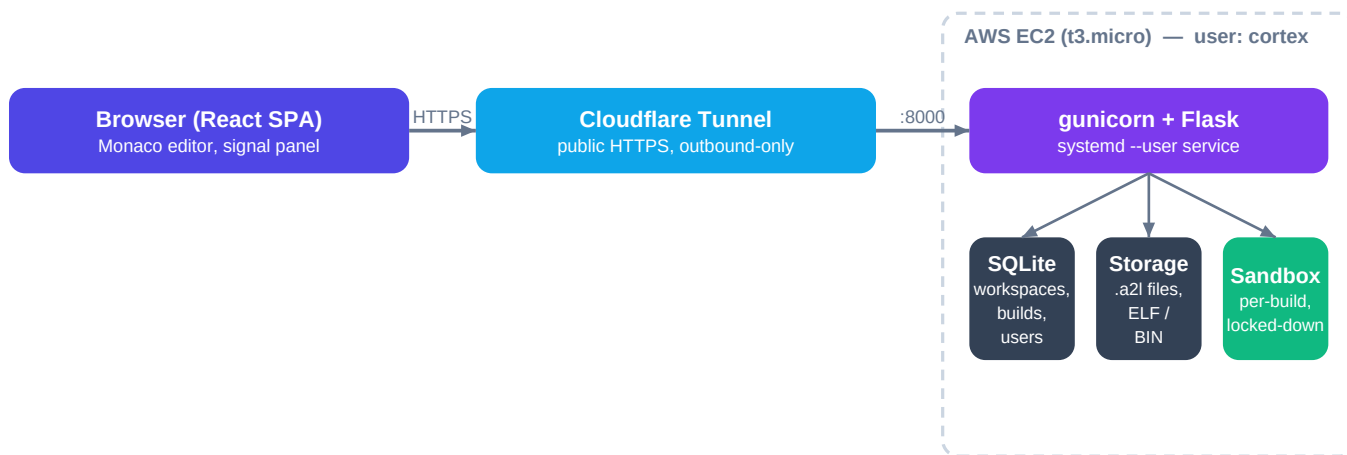


Figure 1 — Request path in production: browser → Cloudflare Tunnel → gunicorn/Flask on EC2, which owns the database, file storage, and the per-build Podman sandbox.

2 Repository map

The four folders this document covers, and how they relate:

backend/	The Flask application: API, database models, A2L parser, header generator, the sandboxed compiler, and the marketplace. Also serves the built frontend.
build/	Fixed embedded-C support files (linker script + startup code) that are compiled alongside every user's C, identical on every build. Never user-supplied.
deploy/	The deploy script run on the server for every release (build frontend, install deps, migrate DB, restart service).
frontend/	The React single-page app: login/signup, workspace list, the editor workspace, and the marketplace pages.

HOW THEY FIT The four folders map cleanly onto the request path: **frontend/** runs in the browser, **backend/** is the server, **build/** supplies the fixed files the sandbox compiles, and **deploy/** is how all of it reaches the EC2 box.

3 backend/ — the Flask server

3.1 Files at a glance

app.py	Local dev entry point. Loads .env, builds the app via the factory, runs Flask's dev server.
wsgi.py	Production entry point for gunicorn (gunicorn wsgi:app). Exists to remove an import ambiguity between app.py and the app/ package so load_dotenv() is never skipped.
seed.py	Idempotent script that creates the two required test accounts; prints generated passwords once.
requirements.txt	Python dependencies (Flask, SQLAlchemy, Flask-Migrate, gunicorn, etc.).
pytest.ini	Registers the custom 'podman' marker so the 6 real-container tests can be excluded from CI with a documented reason.
app/__init__.py	The application factory: wires config, DB, migrations, all Blueprints, health check, and SPA fallback route.
app/config.py	Config object read from environment variables; refuses to start without a SECRET_KEY.
app/models.py	SQLAlchemy models: User, Workspace, A2LFile, Build, Artifact.
app/auth.py	Signup / login / logout / me endpoints and the @login_required decorator.
app/workspaces.py	Workspace CRUD, A2L upload, signals API, source save/load, signals.h endpoint.
app/a2l_parser.py	Line-scanner that extracts MEASUREMENT/CHARACTERISTIC signals from an .a2l file.
app/header_gen.py	Turns parsed signals into signals.h (extern decls) and signals_def.c (storage to link against).
app/compiler.py	Runs the C through arm-none-eabi-gcc inside the locked-down Podman sandbox.
app/builds.py	Build trigger + status API, backed by a bounded ThreadPool worker.
app/marketplace.py	Lists, shows, and serves downloads of successful build artifacts.
app/storage.py	Thin file-storage abstraction (local disk today, swappable for R2).
sandbox/Containerfile	Recipe for the compile sandbox image (Debian + ARM GCC, non-root user).
sandbox/build.sh	The fixed 2-command build script baked into that image (cd /build; gcc; objcopy).

migrations/	Alembic migration scripts, one per schema change.
tests/	Pytest suite: parser, header gen, compiler, and the auth/builds/signals/marketplace APIs.

3.2 Key functions, module by module

app/__init__.py

Function	What it does
<code>create_app(config_class)</code>	Application factory. Creates the Flask app, loads config, initialises the DB and Flask-Migrate, registers the auth / workspaces / builds / marketplace Blueprints, and defines the health check and the SPA-fallback route.
<code>serve_spa(path)</code>	Serves a real file from frontend/dist if it exists, otherwise returns index.html so client-side routing (react-router) works on deep links / refresh.

app/auth.py

Function	What it does
<code>login_required(view)</code>	Decorator that returns 401 unless a user_id is present in the session; guards every protected endpoint.
<code>_get_current_user()</code>	Loads the User row for the session's user_id (or None).
<code>signup()</code>	Validates email + password (min 8 chars), rejects duplicates (409), hashes the password, creates the user, and logs them in.
<code>login()</code>	Verifies email + password hash; on success rotates the session and stores user_id. Same error for bad email or bad password (no user enumeration).
<code>logout()</code>	Clears the session.
<code>me()</code>	Returns the current user; used by the frontend on load to decide if someone is already logged in.

app/models.py

Function	What it does
User	id, email (unique), password_hash, created_at. set_password/check_password wrap Werkzeug hashing; to_dict() never exposes the hash.
Workspace	Owned by a user; holds the source_code and (one-to-one) an A2LFile. Cascade-deletes its A2L file.
A2LFile	One per workspace. Stores the uploaded filename + stored path, plus cached parsed signals/summary as JSON so /signals never re-parses.
Build	One compile attempt: status (queued→running→success/error), duration, exit code, full log, and refs to the ELF/BIN artifacts.
Artifact	One downloadable binary per successful build for the marketplace. Thin: joins through Build for log/duration/user rather than duplicating them.

app/workspaces.py

Function	What it does
<code>_get_owned_workspace(id)</code>	Returns the workspace only if it exists AND belongs to the current user; callers 404 (not 403) on None so other users' workspaces aren't even revealed to exist.
<code>create_workspace()</code> / <code>list_workspaces()</code>	Create a named workspace for the current user; list that user's workspaces, newest first.

<code>get_workspace(id) / delete_workspace(id)</code>	Fetch one workspace (with A2L info); delete it and its stored A2L file.
<code>upload_a2l(id)</code>	Validates extension + size, parses the file, stores the raw bytes, and caches parsed signals + summary. Returns 422 on unparseable input.
<code>get_signals(id)</code>	Returns the cached measurements, characteristics, and summary for the workspace's A2L file.
<code>get_source(id) / save_source(id)</code>	Load and save the workspace's C source (size-capped at 256 KB).
<code>get_signals_header(id)</code>	Generates and returns signals.h as text/plain (the 'View signals.h' link).

app/a2l_parser.py

Function	What it does
<code>parse_a2l(text)</code>	Main entry. Scans line by line for /begin.../end blocks, extracts MEASUREMENT signals fully and CHARACTERISTIC blocks by name only, and returns measurements, characteristics, and a summary (counts, datatypes seen, skipped blocks). Raises <code>ParseError</code> on non-A2L or truncated input.
<code>_parse_measurement_body(line s)</code>	Pulls datatype, conversion method, numeric limits, ECU_ADDRESS, and MATRIX_DIM out of a MEASUREMENT block; reports a skip reason instead of guessing when a field can't be read.
<code>_parse_header(rest) / _to_number(t) / _direction_for(kind)</code>	Helpers: read a block's name + description; parse a token as int/float or None; map MEASUREMENT→input, CHARACTERISTIC→output.

DESIGN CHOICE The parser deliberately fails loudly. CHARACTERISTIC fields are left null (there were none in the reference file to validate against) and every unsupported block is reported in a skipped list rather than faking data.

app/header_gen.py

Function	What it does
<code>generate_header(measurements)</code>	Produces signals.h: one 'extern ;' per MEASUREMENT, mapping A2L datatypes to types. Unsupported types become a comment, not a guess.
<code>generate_definitions(measurements)</code>	Produces signals_def.c: one zero-initialised definition per declared signal so the externs actually link. Uses the same resolver, so identifiers always match signals.h.
<code>sanitise_identifier(name)</code>	Maps an arbitrary A2L name to a legal C identifier (illegal chars → '_', leading digit prefixed). Mirrored on the frontend so clicking a signal inserts the exact name signals.h will declare.
<code>_resolve_signals(measurements)</code>	Shared pass that sanitises + de-duplicates identifiers in order, so the header and the definitions file never disagree.

app/compiler.py

Function	What it does
<code>run_build(source_code, measurements)</code>	The heart of the compile. Writes user.c + generated signals.h/signals_def.c + copies the fixed startup.c/link.ld into a fresh temp dir, runs the sandbox container, captures stdout/stderr, enforces the timeout, and returns status, exit code, capped log, duration, and the ELF/BIN bytes. Never raises on ordinary build failure.
<code>_sandbox_run_args(workdir)</code>	Builds the exact, fully server-controlled 'podman run' argv. Every flag maps to one threat-model line: --network none, --read-only, --cap-drop=ALL, memory/pids/cpu caps, non-root --user, and a single bind mount of the per-build workdir.

app/builds.py

Function	What it does
----------	--------------

<code>trigger_build(workspace_id)</code>	Checks ownership + that there is source, enforces a one-outstanding-build-per-user rate limit (429), creates a queued Build, and submits the job to the worker pool. Returns the build id immediately.
<code>_run_build_job(app, build_id)</code>	Runs in a worker thread with its own app context: marks the build running, gathers source + signals, calls <code>run_build()</code> , stores artifacts + log, and (on success) creates the marketplace Artifact.
<code>get_build(id) / list_builds(workspace_id)</code>	Poll a single build's status/log (owner-only); list a workspace's recent builds.

app/marketplace.py & app/storage.py

Function	What it does
<code>list_marketplace()</code>	Returns every Artifact (all successful builds, all users), newest first.
<code>get_artifact(id) / download_artifact(id)</code>	Show one artifact's metadata + full build log; stream the raw binary as an attachment.
<code>LocalStorage.write_bytes / read_bytes / read_text / delete</code>	Disk-backed file store keyed by a relative path; the single seam where Cloudflare R2 could later be swapped in without touching callers.

4 build/ — fixed embedded-C support files

These two files are compiled alongside every user's C, unchanged on every build, and are never user-supplied. They give the user's `main()` a minimal, self-consistent Cortex-M4 environment to link against.

<code>startup.c</code>	The boot/reset code: defines the interrupt vector table (initial stack pointer + <code>Reset_Handler</code> + the ARMv7-M fault/exception handlers), then <code>Reset_Handler</code> copies <code>.data</code> from flash to RAM, zeroes <code>.bss</code> , and calls <code>main()</code> . Fixed, checked into the repo.
<code>link.ld</code>	The linker script: a generic Cortex-M4 memory map (512K FLASH at 0x08000000, 128K RAM at 0x20000000) and the section layout (<code>.isr_vector</code> , <code>.text</code> , <code>.data</code> , <code>.bss</code>). Internally consistent for the linker; nothing flashes real hardware.

WHY FIXED Keeping `startup.c` and `link.ld` server-side and fixed is a security decision (threat T7): the user controls only the body of their C program, never the compiler flags, filenames, linker script, or entry point.

5 deploy/ — shipping to the server

<code>server-deploy.sh</code>	The one script every deploy runs on the EC2 box, in order: (1) <code>npm ci && npm run build</code> in <code>frontend/</code> , (2) activate <code>venv</code> + <code>pip install -r requirements.txt</code> in <code>backend/</code> , (3) source <code>.env</code> and run <code>flask db upgrade</code> , (4) <code>systemctl --user restart cortex-studio</code> . It assumes the repo is already pulled to the latest commit — GitHub Actions does the git pull before invoking it.
-------------------------------	---

6 frontend/ — the React SPA

6.1 Files at a glance

<code>src/main.jsx</code>	Entry point: mounts into <code>#root</code> inside <code>BrowserRouter</code> (client-side routing).
<code>src/App.jsx</code>	Top-level router. Fetches the current user once on load, then guards <code>/workspaces</code> , <code>/marketplace</code> , etc. behind a <code>RequireAuth</code> wrapper that redirects to <code>/login</code> .
<code>src/api.js</code>	One place wrapping every backend call: a <code>fetch()</code> helper that always sends the session cookie (credentials: include) and throws the backend's error message on non-2xx.
<code>src/pages/Login.jsx / Signup.jsx</code>	The auth forms; on success they set the app-level user and navigate in.

<code>src/pages/WorkspaceList.jsx</code>	Lists the user's workspaces and creates new ones; shows whether each has an A2L uploaded.
<code>src/pages/Workspace.jsx</code>	The main IDE screen: Monaco editor, signals panel, Upload A2L / View signals.h / Save / Compile, and the live build console.
<code>src/pages/Marketplace.jsx</code>	Table of every successful build artifact (filename, workspace, user, time, duration, size).
<code>src/pages/ArtifactDetail.jsx</code>	One artifact's detail + full compile log + a download link.
<code>src/App.css / index.css</code>	Styling. vite.config.js configures the dev server + proxy and the production build.
<code>dist/</code>	The built SPA (HTML/CSS/JS) that Flask serves directly in production.

6.2 Key functions / behaviour

`src/api.js`

Function	What it does
<code>request(path, options)</code>	Core fetch wrapper: base '/api', sends cookies, parses JSON, throws data.error on failure.
<code>login / signup / logout / getCurrentUser</code>	Auth calls; getCurrentUser returns null on 401 instead of throwing, so the app can boot logged-out.
<code>listWorkspaces / createWorkspace / getWorkspace / deleteWorkspace</code>	Workspace CRUD.
<code>uploadA2L(id, file)</code>	Multipart POST of the .a2l file (separate from the JSON wrapper).
<code>getSignals / getSource / saveSource / signalsHeaderUrl</code>	Signals + source read/write, and the URL for the raw signals.h.
<code>triggerBuild / getBuild</code>	Start a compile and poll its status.
<code>listMarketplace / getArtifact / artifactDownloadUrl</code>	Marketplace list, detail, and binary download URL.

`src/pages/Workspace.jsx`

Function	What it does
<code>handleUpload(e)</code>	Uploads the chosen .a2l, then refreshes the workspace + signals so the panel populates.
<code>loadSignals()</code>	Fetches the signal set for the workspace; shows a friendly error if none is uploaded yet.
<code>handleSave()</code>	PUTs the current editor contents; shows a transient Saving.../Saved state.
<code>handleCompile()</code>	Saves first (so stale code isn't built), triggers a build, then starts polling.
<code>pollBuild(id)</code>	Polls GET /builds/ every 800 ms while queued/running, updating the console live.
<code>insertSignal(signal)</code>	Inserts the sanitised identifier at the cursor — the click-to-insert on the signal panel.
<code>sanitiseIdentifier(name)</code>	Client copy of the backend rule so the inserted name matches signals.h exactly.

WHY POLLING Compile uses **polling, not streaming**: gcc finishes sub-second for realistic inputs and run_build only returns once the container exits, so there is no partial output to stream. The brief explicitly allows this fallback.

7 How the pieces work together

Every user action follows the same four layers. A React component calls a function in `api.js`, which hits a Flask Blueprint route, which is guarded by `@login_required` and an ownership check before it touches a model or the compiler.

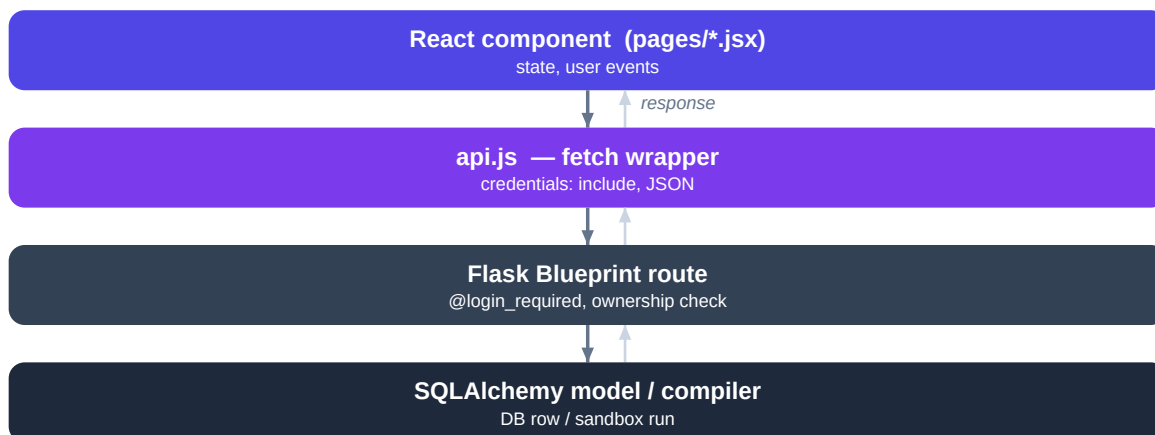


Figure 2 — The request/response layering shared by every feature.

7.1 Authentication flow

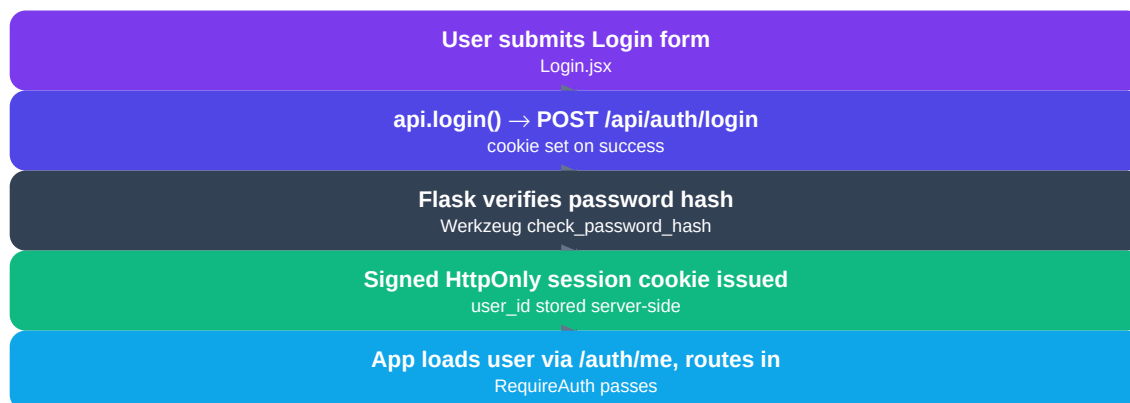


Figure 3 — Login. Sessions are server-side; the cookie only carries a signed `user_id`, never credentials.

7.2 A2L upload → signals → header

This is how an uploaded ECU description turns into things the C code can actually reference:

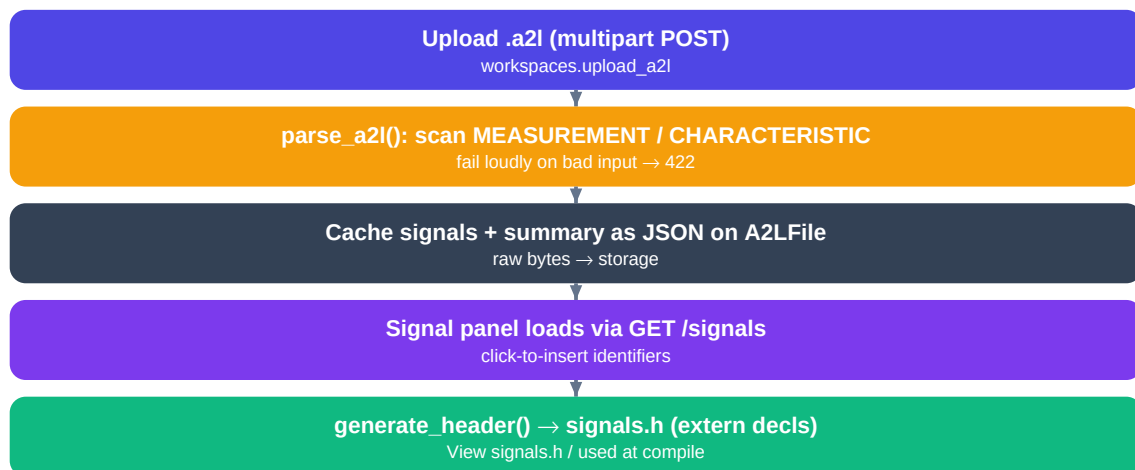


Figure 4 — Chosen approach for 'how signals reach the code': a generated header file (justified over accessor stubs or -D injection).

7.3 The compile pipeline

The most important flow in the app, and the one with the real security surface. The frontend and backend run as two parallel tracks joined by the build id:

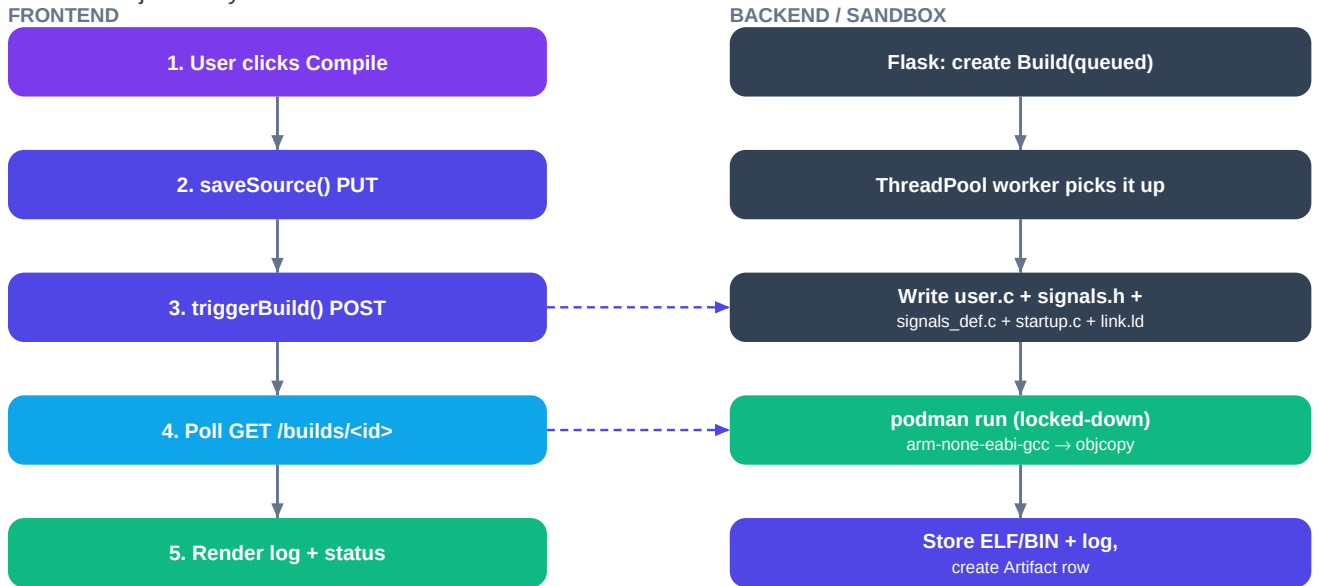


Figure 5 — Compile: the frontend saves → triggers → polls; a bounded worker pool runs each build in a throwaway sandbox and records the artifact.

7.4 Inside the sandbox (the threat model)

Compiling a stranger's C is the sharp edge of the assignment. Each build gets a brand-new container that can do almost nothing except run the fixed build script:

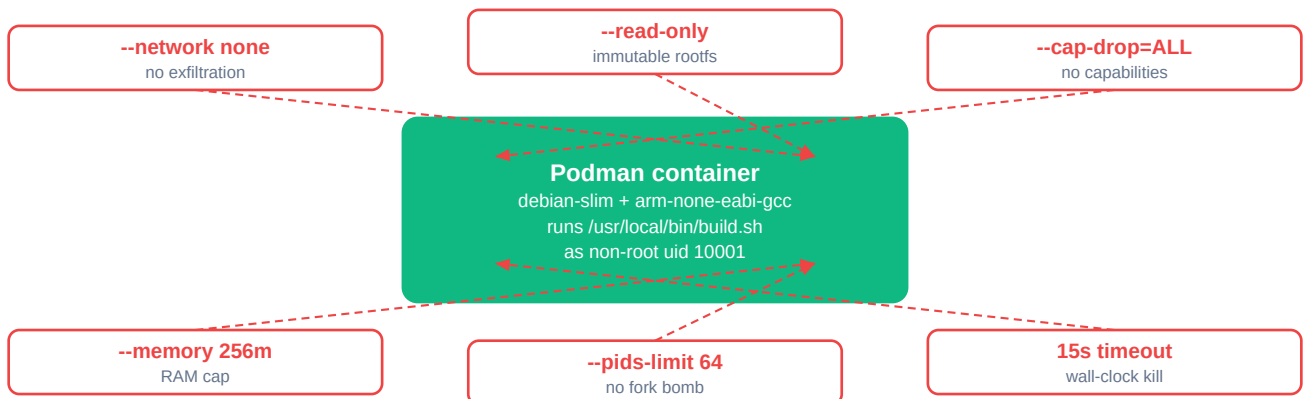


Figure 6 — Every 'podman run' flag maps to one line of the documented threat model. The user controls only the body of user.c.

7.5 Marketplace flow

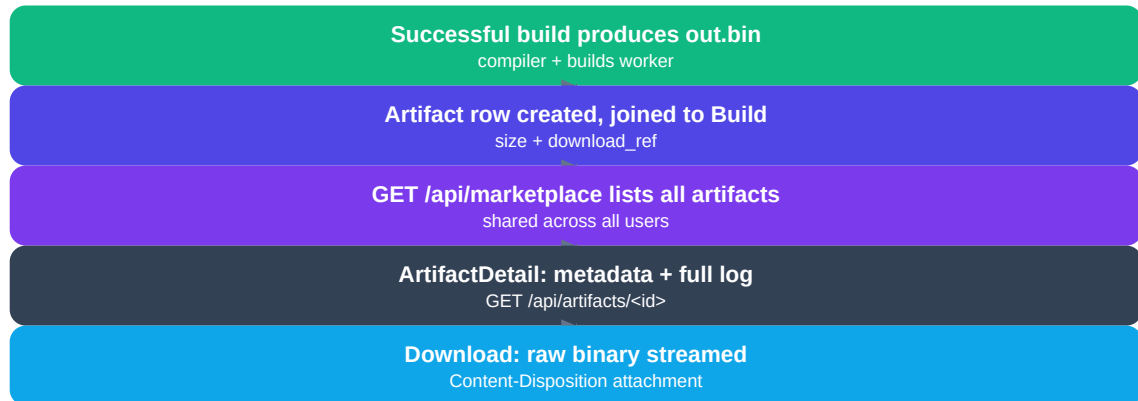


Figure 7 — Marketplace: every successful build is browsable and downloadable by any logged-in user (the chosen visibility model).

8 Production workflow & deployment

One AWS EC2 t3.micro runs everything as a sudo-less cortex user: gunicorn/Flask under systemd serves the built SPA and the API; each compile spawns a rootless Podman container; a Cloudflare Tunnel gives a public HTTPS URL without opening any inbound web port.

8.1 CI/CD pipeline (GitHub Actions)



Figure 8 — Push-to-deploy. The 6 real-container compiler tests are excluded from CI (rootless-Podman/cgroup mismatch on shared runners) with the reason documented, not silently skipped.

8.2 The security posture, in layers

Layer	Choice	Why
Network	Cloudflare Tunnel	No inbound 80/443/8000 open; only SSH/22. App binds 127.0.0.1 only.
OS user	sudo-less 'cortex'	App + CI + sandbox all run as a user that can't escalate to root.
Per compile	Rootless Podman	--network none, --read-only, --cap-drop=ALL, mem/pids/cpu caps, 15s kill.
Input	Server-controlled argv	User controls only user.c's body — never flags, filenames, or linker script.
Throughput	Bounded worker pool	ThreadPool + 1-build-per-user limit, so a burst queues instead of forking containers.

STATED HONESTLY Honest, documented limitations: SQLite + local disk on a single instance means data is lost if the box is terminated; the Quick Tunnel URL changes on restart; the public IP isn't yet an Elastic IP. All are accepted free-tier tradeoffs, written down rather than hidden.

9 Questions an interviewer is likely to ask

The brief says every line must be defensible and that consciously-cut scope will be probed, not penalised. These map to the assignment's own requirement areas.

Q Compiling a stranger's C on your server is the sharp edge. What's your threat model?

A Each build runs in a fresh, rootless Podman container with `--network none` (no exfiltration/callbacks), `--read-only rootfs` with a single writable per-build bind mount, `--cap-drop=ALL + no-new-privileges + non-root uid` (no privilege escalation), and memory/pids/cpu caps plus a 15-second wall-clock kill (no resource exhaustion or infinite loops). The user controls only the body of `user.c` — never the compiler flags, filenames, entry point, or linker script, which are fixed server-side. What's still open: I trust the gcc toolchain itself and the host kernel's container isolation; a kernel-level container escape is out of my mitigation.

Q How is authentication stored and validated, and what's a workspace's isolation boundary?

A Passwords are hashed with Werkzeug's `generate_password_hash` (PBKDF2) — never stored in plaintext. Auth is a Flask server-side session: the cookie is `HttpOnly, SameSite=Lax, Secure` in production, and only carries a signed `user_id`; the session is rotated on login/signup. The isolation boundary is ownership: every workspace/build/source route calls `_get_owned_workspace` and 404s (not 403s) if the row isn't owned by the session user, so you can't even confirm another user's workspace exists. Before real users I'd add CSRF tokens, email verification, rate-limited login, and move sessions to a shared store.

Q Why a generated header for signals, over accessor stubs or -D injection?

A A header is the most transparent, ordinary way for C to see externally-defined symbols — it's how embedded devs already work with memory-mapped signals. `generate_header` emits one 'extern' per MEASUREMENT; `generate_definitions` emits a matching zero-initialised definition so the externs actually link. Accessor stubs would invent an API the user didn't ask for; -D injection would be invisible in the editor. I rejected both for being less faithful to how real firmware is written.

Q Your A2L parser is not a full ASAM parser. What did you cut and why?

A The brief explicitly says a full parser isn't expected. I scan MEASUREMENT blocks fully (datatype, limits, address, `matrix_dim`) and CHARACTERISTIC blocks by name/kind only — the reference file had zero CHARACTERISTIC instances to validate field layout against, so rather than guess field positions I leave them null and report every unsupported block in a 'skipped' list. That's the ground rule: fail loudly, never fake data. Unparseable input returns a 422 with a specific reason.

Q Streaming vs polling for compiler output — why polling?

A The brief allows polling as a fallback. gcc runs sub-second for realistic inputs and `run_build` only returns once the container exits, so there's genuinely no partial output to stream mid-build. The frontend polls `GET /builds/` every 800 ms and renders the full log + status + exit code + duration — warnings and errors included, not just a pass/fail badge. Streaming would add complexity for output that arrives all at once anyway.

Q What's your marketplace visibility model, and why?

A Every successful build's artifact is browsable and downloadable by any logged-in user — the simplest of the three models the brief allows, and it matches the brief's own words, 'a shared, browsable catalogue.' Each entry carries the binary, the full compile log, creation time, build duration, and the originating workspace + user. Artifact stays thin and joins through Build rather than duplicating that metadata.

Q How do you stop one user from hammering the compiler?

A Two layers: a `ThreadPoolExecutor` with a small fixed worker count so concurrent builds queue instead of spawning unbounded containers, and a per-user rate limit — at most one queued/running build at a time, else 429. I chose the simple one-outstanding rule over a token bucket because at this scale it's enough and obvious to reason about.

Q Why SQLite and local disk? Isn't that fragile?

A Yes, and it's a documented tradeoff: simplicity over durability for a free-tier single-instance deployment. If the EC2 box is terminated, the DB and uploads go with it. The storage layer is a single class (`LocalStorage`) with a tiny interface precisely so a Cloudflare R2 backend and a managed Postgres can be swapped in later without touching callers. I'd make that switch before this held anything I couldn't lose.

Q Walk me through what happens the moment I click Compile.

A The frontend saves the buffer first (so stale code isn't built), POSTs to create a Build row that returns immediately as 'queued', then polls. A worker-pool thread marks it 'running', writes user.c plus the generated signals.h/signals_def.c and copies the fixed startup.c/link.ld into a fresh temp dir, and runs the locked-down container: arm-none-eabi-gcc then objcopy. Output, exit code, and duration are captured; on success the ELF + BIN are stored and a marketplace Artifact is created. The poll then shows the final log.

Q How is the frontend served, and how does routing survive a refresh?

A One Flask process serves both. In production the built SPA lives in frontend/dist and Flask's serve_spa returns a real file if it exists, else index.html — so a deep link like /workspaces/5 or a hard refresh still loads the app and react-router takes over client-side. In dev, Vite serves the SPA and proxies /api to Flask.

Q What would you change or add with more time?

A Elastic IP + a named Cloudflare Tunnel on a fixed hostname (the Quick Tunnel URL changes on restart); Postgres + R2 for durability; CSRF + email verification + login rate-limiting; artifact retention/cleanup; and getting the 6 real-container compiler tests running in CI (they pass on the real box but hit a rootless-Podman/cgroup mismatch on GitHub's shared runners).

Q The brief wanted incremental commits and defensible code. How did you work?

A Committed in phases (parser/API, then compiler/sandbox, then marketplace, then devops/CI) rather than one drop, with the reasoning for each design decision written down in docs/DECISIONS.md and the threat model in docs/SECURITY.md. AI assistance was used, but every file here is one I can walk through and defend line by line, which is exactly what this document is for.

MAPPING Every answer above traces to a requirement in the assignment brief (auth §4.1, workspaces §4.2, A2L §4.3, signals API §4.4, editor §4.5, compilation §4.6, marketplace §4.7) and to the security posture the brief asks to be stated honestly.